Custom Network IDS + Machine Learning

CodeAlpha Cyber Security Internship - Task 4

Project: Signalwatch. **Date:** 29 October 2026. **Implementation:** Python 3.12, Scapy, scikit-learn, SQLite, Flask, Docker Compose.

Executive summary

This project implements our own network detection pipeline: packet decoding, event-time feature extraction, custom rules, Random Forest scoring, incident persistence, console notifications, and a local read-only dashboard. Scapy and scikit-learn are libraries; detection logic and operational processing are our code.

The system was verified with offline fixtures and continuously captured traffic inside an isolated Docker lab. Successive batches produced no benign-control alerts, all five custom signatures, and a separate ML alert without restarting the sensor. Automated tests cover parsing, reassembly, features, labels, model integrity, storage, APIs, browser interactions, and offline integration.

The frozen classifier achieves 95.43% F1 on the selected public test windows and 99.87% F1 on generated held-out lab sessions. These figures must be read with their limits: **the public false-positive rate is 15.41%, and the always-suspicious majority baseline has a higher public F1 of 96.71%** because the public test is heavily imbalanced. The model improves benign rejection, with balanced accuracy 88.40% versus the baseline's 50%, but remains a research/training classifier.

The report distinguishes public benchmark evaluation, generated-session evaluation, and live demonstrations. No universal detection claim, calibrated attack probability, automatic containment, or production certification is made.

Architecture and data flow

The Docker target serves fixed mock HTTP and DNS responses. The sensor shares its network namespace, allowing it to observe target traffic rather than relying on a bridge interface that cannot see unrelated unicast flows. A separate generator produces bounded local scenarios. The lab network is internal; the dashboard publishes only at 127.0.0.1:8088.

```
Lab generator -> mock target interface -> Scapy packet capture
                                       -> custom protocol/rate rules
                                       -> five-second flow features -> Random Forest
                                       -> merged alert windows -> SQLite incidents
                                       -> console + read-only dashboard + exports
```

Rules can alert immediately. Model inference occurs when a five-second feature window closes. The dashboard refreshes every two seconds. Capture callbacks enqueue packets without retaining full capture history; the main processing loop handles detection and persistence. Queue capacity is 2,048 packets and active feature capacity is 10,000 windows. Dropped, late, unsupported, and

stream-limited traffic is observable in status.

Stored alerts retain source/destination metadata, rule IDs, severity, model score, and numerical feature snapshots; they do not store submitted credential values or raw payload bodies. Rule and model evidence for the same run/window merge into a single event. Incident IDs group a run's source/destination pair into fixed 60-second event-time buckets. Counts represent unique alert windows, not unique attackers or confirmed compromises.

SQLite WAL supports a writer and read-only dashboard connections. Each alert update is transactional. Identical offline PCAP replay uses a content-derived run ID and stable window IDs, preserving idempotence. Live restarts retain stored incidents but begin a new capture session; packets during downtime cannot be recovered. Heartbeats distinguish active, stale, stopped, failed, and completed-replay states.

Detection rules and response behavior

ID	Condition	Severity	Response
CUSTOM-001	ICMP echo request, used as a connectivity test	Informational	Verify the intended source and test; do not call this an attack.
CUSTOM-002	More than 10 TCP SYN packets without ACK from a source to a destination within 5 seconds	Medium	Investigate fan-out, retries, and authorized scans.
CUSTOM-003	A complete form-encoded HTTP POST body contains a password field	Medium	Identify the service and require HTTPS; investigate real exposure if applicable.
CUSTOM-004	A request path contains a traversal segment after at most two URL decodings	High	Review routing, path normalization, access controls, and related server logs.
CUSTOM-005	DNS query exactly matches blocked.lab.example	Medium	Inspect the originating process; this reserved name is a training indicator.
ML	Suspicious-class score meets the frozen 0.95 threshold	Medium	Inspect surrounding activity and feature evidence before containment.

The SYN detector uses a bounded sliding timestamp deque. The HTTP path uses our first-seen TCP reassembly with at most 16 pending segments, 64 KiB per stream, 128 streams, and 30-second idle expiry. Tests cover split requests, out-of-order segments after a known SYN, retransmission, completed request bodies, negative controls, and capacity limits.

Rules are configured through JSON thresholds, fixed supported detection kinds, stable IDs, and response text. There is no expression evaluation or shell command execution based on packet contents. Alert notifications remain local. Human-readable playbooks accompany incidents; network blocking was not selected for this project.

Protocol coverage is deliberately bounded. This implementation is not a complete TCP/IP inspection stack. It does not decrypt TLS, fully parse HTTP/2 or chunked/compressed request bodies, reassemble IP fragments, or handle every TCP overlap/sequence-wrap evasion. Capture and parser limits can cause misses and are documented rather than hidden.

Machine learning data and methodology

Public subset and label quality

The project analyzes seven selected IoT-23 captures and their official connection labels. Attribution: Garcia, Parmisano, and Erquiaga (2020), DOI 10.5281/zenodo.4743746. The published Zenodo record specifies CC-BY-4.0; historical source metadata/notices are retained in the model card. Raw dataset files are not redistributed in Git or the ZIP.

Partition	Public captures	Independent generated lab sessions
Training	Malware 8-1 and 20-1	12 sessions, seeds 100–111
Validation	Malware 21-1 and benign Honeypot 4-1	5 sessions, seeds 300–304
Test	Malware 3-1 and 42-1; Honeypot 7-1/Somfy-01	6 sessions, seeds 600–605

Whole capture/session groups are assigned before training. No random row split places windows from the same source session into both training and test. Raw addresses, raw port numbers, capture identifiers, absolute timestamps, payload markers, rule results, and labels are excluded from learned features.

Official labels are joined using bidirectional endpoints, protocol, and time. A window must fit exactly one connection interval, allowing 1 ms before start and 1 second after end for timestamp precision and final teardown. Unknown/unmatched/ambiguous windows are excluded. The generator's declared scenario intent supplies lab labels, independently of detector outputs. An infected capture is not assumed malicious in its entirety.

The source/lock manifests record URLs, partitions, sizes, and SHA-256 values. Data-quality evidence records matched classes, excluded windows, unsupported packets, late packets, and capacity drops by capture. One source capture contains a truncated record; unsupported/malformed cases remain part of the recorded quality accounting.

The selected downloads total 100,847,227 bytes. They contain 675,440 packets; 38,926 packets are unsupported/malformed under this decoder. Public extraction yields 243,827 matched labeled windows and excludes 14,823 unmatched windows across all partitions. No late-packet or flow-capacity drops were recorded during this extraction.

Shared feature and training contract

Schema flow5-v1 has 22 features: duration, directional packet counts and IP-layer bytes, packet-length moments/range, inter-arrival moments, packet/byte rates, SYN/RST/ACK/FIN fractions, protocol indicators, and per-source connection/destination-port diversity.

Windows use floor(packet_timestamp / 5), with direction from the first observed packet. Rates use a minimum 1 ms denominator. Packet statistics are population moments. The same packet decoder and extractor run during public-data preparation, lab generation, offline analysis, and live capture. MAC addresses do not enter the feature values.

Training uses 200 trees, maximum depth 10, a maximum of 256 leaves, balanced bootstrap class weights, random seed 42, and two workers. Threshold candidates are 0.05 through 0.95 at 0.01 intervals. Validation F1 selects 0.95, using the higher threshold for ties. Parameters and threshold were frozen before inspecting test results and were not adjusted afterwards.

The artifact is approximately 190 KB. Metadata records its hash, feature order, exact scikit-learn version, threshold, parameters, feature importances, dataset-lock hash, and measured CPU inference latency. Model loading checks these contracts. joblib uses pickle: the artifact and metadata must be trusted project files; a hash is not a sandbox for untrusted models.

Held-out evaluation

Dataset	Precision	Recall	F1	False-positive rate
Public subset	98.87%	92.22%	95.43%	15.41%
Generated lab sessions	100.00%	99.73%	99.87%	0.00%

Public test: 187,534 windows, including 11,957 benign and 175,577 suspicious. Average precision (PR-AUC convention used here) is 98.02%. Lab test: 677 windows, including 305 benign and 372 suspicious; average precision is 100%.

Partition	True negative	False positive	False negative	True positive
Public	10,114	1,843	13,664	161,913
Lab	305	0	1	371

The majority baseline predicts suspicious for every test window: public F1 96.71%, public false-positive rate 100%, lab F1 70.92%, and lab false-positive rate 100%. The public model's balanced accuracy is 88.40%, compared with 50% for the baseline. This explains why a single F1/accuracy headline is inadequate: the model rejects more benign traffic but still generates many false alarms and misses some malicious windows.

Generated sessions are simpler than public data. Public windows are correlated, especially where long connections produce many windows. These are subset-level measurements, not independent-flow confidence intervals or results for the full IoT-23 dataset. Dataset age, IoT device patterns, label-join exclusions, and deployment drift limit generalization.

Verification and operational evidence

Automated tests

The Python suite covers known feature values, offline/stream equivalence, fixed-window timing, flow limits, payload-identity exclusion, IPv6/ICMP decoding, fragments, rule thresholds, HTTP reassembly, DNS matching, model/schema/hash checks, invalid feature matrices, label ambiguity, partition separation, storage merging, restart persistence, API validation, read-only endpoints, stale health, all five offline signatures, negative controls, and idempotent replay.

The browser suite checks displayed incident data, literal rendering of adversarial source strings, filters, detector details, model-card disclosure, incident navigation, new alerts appearing through polling, keyboard access, and layouts at 320, 390, 768, and 1440 pixels. Temporary browser servers and databases are owned and removed by the tests.

Test transcripts, dependency-audit JSON, live evidence, code fingerprints, and final results are saved under reports/evidence. Passing tests establish the documented cases, not exhaustive detection coverage. GitHub Actions checks the packaged model and offline/browser behavior; public-data training remains a separately reproducible operation.

The final local suite has **30 Python tests and five browser checks**. The recorded dependency audit covers all 48 pinned runtime/development packages and reports no known advisories at scan time. The final verification manifest identifies the exact checked source commit and file hashes.

Continuous live verification

The live verifier runs three successive batches against the same sensor run ID:

- Benign HTTP/DNS: no rule alerts and no ML flags in the recorded verification.
- Combined suspicious scenarios: all five custom signatures; no ML flags at the frozen threshold.
- Separate variable-size UDP burst: a live ML alert, with its actual score recorded in live_capture.json.

The sensor remains active throughout; queue/flow/stream losses and heartbeat are recorded. The variable-size demonstration is not part of the held-out benchmark. The missed fixed-size traffic is retained as an observed limitation rather than relabeled a success. The model was not retrained or threshold-adjusted for the demonstration.

Live dashboard screenshots show actual Docker detections. The interface distinguishes current-run packet/model counters from all stored alert/incident counts. Incident-table aggregation is bounded to the latest 500 matching alert windows, with that limit displayed. Capture status is never inferred from an absence of alerts.

Reproduce

```
python -m pytest -q
npm run test:browser
python tools/verify_live.py
```

See the README for prerequisite installation, browser setup, the pinned Docker build, startup, and data/training commands. Verification records the actual versions and image ID; do not treat an unexecuted workflow file as evidence of a successful hosted run.

Operating instructions and limitations

Start with docker compose build target, followed by docker compose up -d --no-build. Open http://127.0.0.1:8088/ and wait for Capture active. Generate only the named lab scenarios. The sensor needs NET_RAW in its target namespace; the target needs low-port binding for mock DNS. The app does not receive a Docker socket or host-network access.

Use docker compose down to stop the owned lab, preserving its named incident volume. The native Linux capture command in the README requires an authorized interface and raw-socket permission; the separately verified live mode is Docker, not an untested claim about a host network.

Missing or incompatible model files cause an explicit degraded ML state while custom rules continue. Read-only APIs validate filters and bound returned rows; packet-derived browser values use text rendering. Runtime logs, SQLite files, private captures, credentials, and downloaded public data are excluded from Git and the source ZIP.

Future work should use independently labeled deployment traffic and a new untouched test partition, broaden normal traffic diversity, assess calibration/operating thresholds, extend protocol reassembly, and measure load/drops under realistic throughput. This prototype does not provide production authentication, Internet-facing hosting, central incident workflow, or firewall containment.

Sources and submission

See SOURCES.md for dataset attribution and library references, MODEL_CARD.md for complete model disclosures, and DEMO_AND_SUBMISSION.md for a three-minute video outline and the internship checklist. The public repository contains the source, trusted model, synthetic examples, reports, and evidence; LinkedIn posting and internship-form submission remain manual.